Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Phases of compiler

Class Notes

Unit No: 2
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 1

Introduction to Compiler.

A compiler is a software that typically takes a high level language (like C++ and java) code as input & converts the inputs to a lower level language at once. It lists all the errors if the input code does not follow the rules of its language. This process is much faster than interpreter but it becomes difficult to debug all the errors together in a program.

→ A program which is input to compiler is called as source program.

Phases of compiler

A compiler operates in phases. A phase is logically integrated operation that takes source program in 1 representation & produces output in another representation.

The phases of compiler are shown below!

i) Analysis (Machine Independent / language dependent)
ii) Synthesis (Machine dependent / language independent)

→ Compilation process subdivided into number of sub processes called 'Phases'.

Lexical Analysis

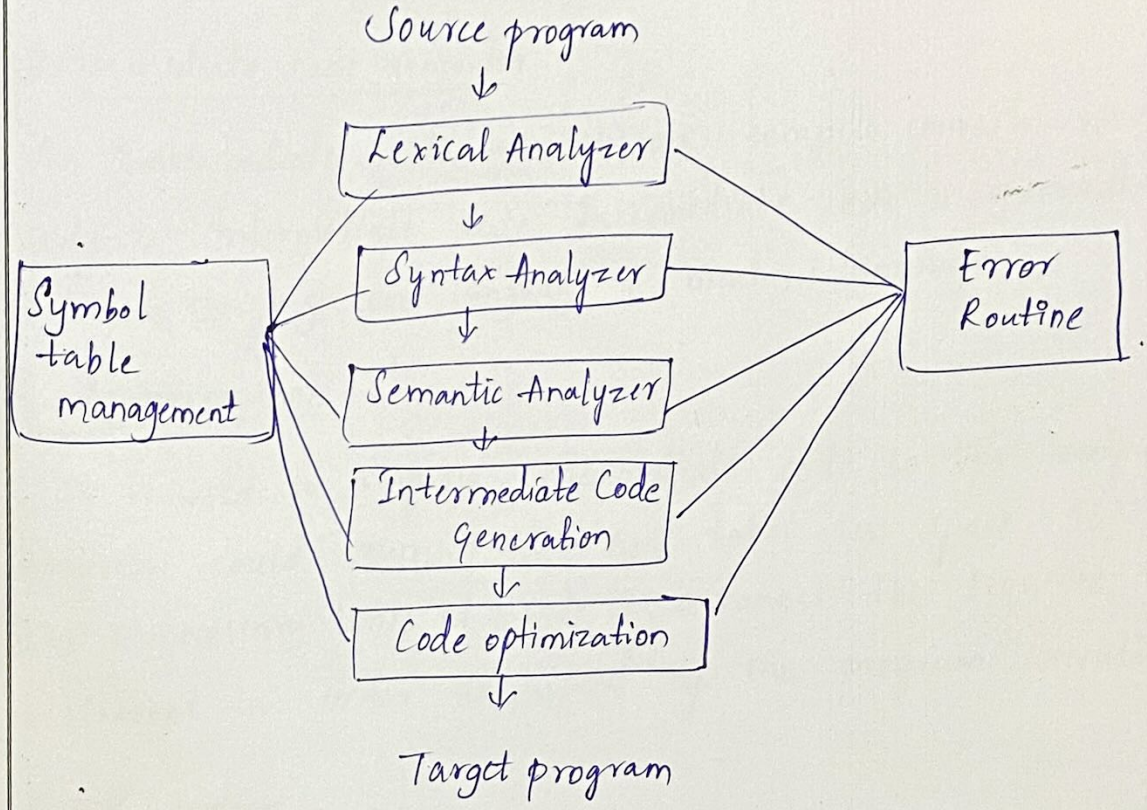
* It is also called as Scanner.
* Lexical analyzer phase is 1st phase of compilation

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Phases of compiler

Class Notes

Unit No: 2
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 2

process. It takes source code as input. It reads source program one character at a time & converts it into meaningful lexemes. lexical analyzer represents these lexemes in forms of tokens



Syntax Analyzer: (Parser)

Syntax analyzer takes tokens as input and generates a parse tree as output. In syntax analysis phase, the parser checks that expression made by tokens is syntatically correct or not.

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Phases of compiles

Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 3

Semantic Analysis:

It checks whether the parse tree follows rules of language semantic analyzer keeps track of identifiers, their types & expressing. The output of semantic analysis phase is annotated tree syntax.

Intermediate code generator

An Intermediate code generation, compile converts source code to Intermediate code. Intermediate code is generated between the high level language & machine language.

Code optimization

It is used to improve intermediate code so that output of program could run faster and takes less space. It removes the unnecessary lines of code & arranges the sequence of statement in order to speed up the program execution.

Code Generation:

It takes the optimized intermediate code as input & maps it to target machine language. It translates the intermediate code into the machine code of the specified compiler.

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Phases of compiler

Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 4

Ex :-

Sum := oldsum + Rate * 50

↓

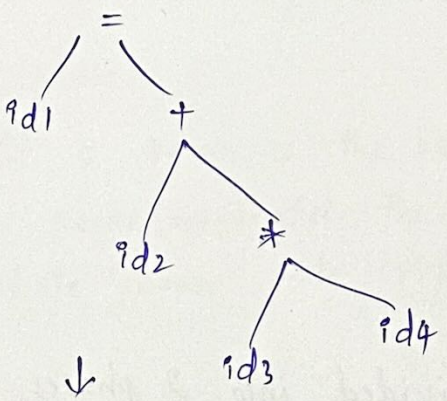
Lexical Analyzer

id1 = id2 + id3 * id4

↓

Syntax analyzer

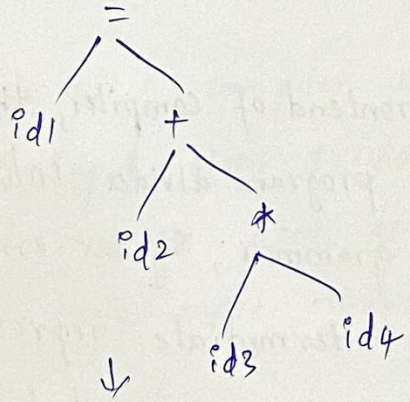
↓



↓

Semantic analyzer

↓



↓

Intermediate code

temp1 := int to real (50); temp2 := id3 + temp1

temp3 := id2 * temp2, id1 = temp3

↓

code optimizer

temp1 := id3 * 50.0, id1 = id2 + temp1

↓

code generator

```
MOVF    id3.R2
MULF    #50.0.R2
MOVF    id2.R1
ADDF    R2.R1
MDVF    R1.id1
```

Pass and Phases of Translation

A compiler can broadly divided into 2 phases based on way they compile:

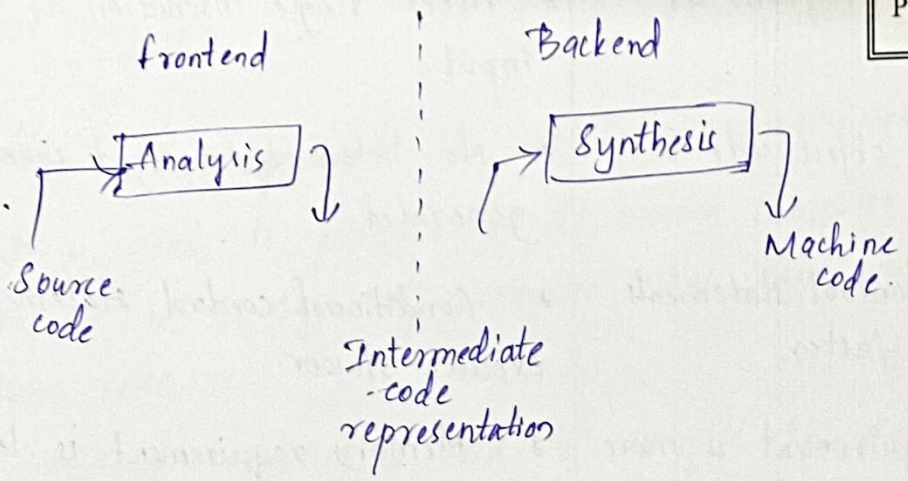
Analysis phase:

It is also known as frontend of compiler, the analysis of the compiler reads source program divides into core parts and then checks for lexical, grammar, syntax errors. the analysis phase generates the intermediate representation of the source program, and symbol table, which should be fed to the synthesis phase as input.

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Pass and phases of Translation.

Class Notes

Unit No: 2
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 5



Synthesis phase:

It is known as the back end of the compiler, the synthesis phase generates the target problem with help of intermediate source code representation & symbol table.

Pass:

It refers to traversal of a compiler through the entire program.

Phase:

Phase of a compiler is a distinguishable stage, which takes input from previous stage, processes and yields output that can be used as input for next stage.
A pass can have more than one phase.

Compiler	Interpreter
* Takes entire program as input.	* Takes single instruction as input.
* Intermediate object code is generated	* No intermediate object code is generated.
* Conditional control statements are executed faster	* Conditional control statements execute slower.
* Memory requirement is more	* Memory requirement is less.
* Program no need to be compiled every time	* Everytime higher level program is converted into lower level.
* Errors are displayed after entire program is checked.	* Errors are displayed for every instruction interpreted.
* C, C++ use compilers	* Python, Ruby etc are interpreters.

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Boot Strapping

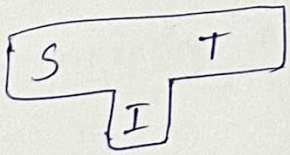
Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 6

Boot Strapping:

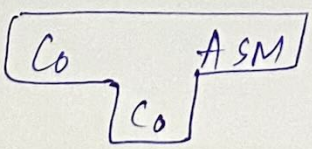
→ It is widely used in compilation development.
→ It is used to produce a self hosting compiler. Self-hosting compiler is a type of compiler that can compile its own source code.
→ Bootstrap compiler is used to compile the compiler and then you can use this compiled compiler to compile everything else as well as future versions of itself
→ A compiler can be characterized by 3 languages:

* Source language
* Target language
* Implementation language



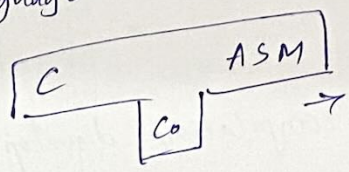
Compiler which takes C language & generates an assembly language as an output with the availability of a machine of assembly language.

Step1: First we write a compiler for small of C in assembly language.

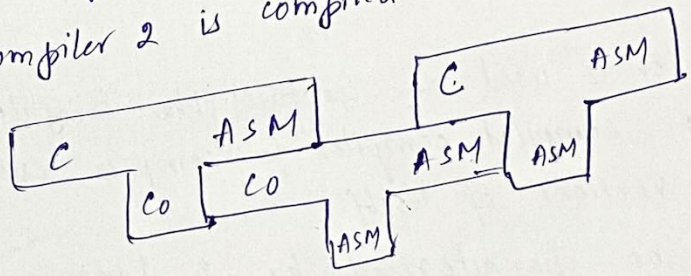


→ Compiler 1

Step2: Then using the small subset of C i.e., Co for source language C the compiler is written



Step3: finally we compile the 2nd compiler, using compiler 1 the compiler 2 is compiled



Step4: Thus we get a compiler written in ASM with ~~which~~ compiler C & generates code in ASM

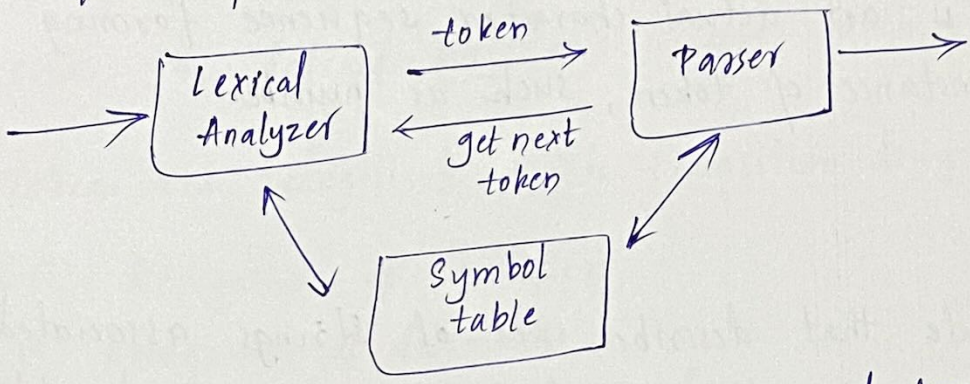
Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: The Role of Lexical Analyzer

Class Notes

Unit No: 2
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 7

The Role of lexical Analyzer

The lexical Analyzer is the first phase of a compiler. Its main task is to read the input character & produce as output a sequence of tokens that parser uses for syntax analysis.



→ Upon receiving a 'get next token' command from parser, the LA reads input character until it can identify next token.

→ Lexical Analysis is a process of taking an input string of characters & producing a sequence of symbols called lexical tokens

1) LA eliminates comments & white spaces in the form of blanks, tab space & newline characters.

2) Correlate error messages from compiler with the source program.

Token!

A token is a group of characters having collective meaning: typically a word or punctuation mark; seperated by a lexical analyzer and passed to a parser (syntax analyzer)

→ A lexem is an actual character sequence forming a specific instance of token, such as number.

Pattern!

A rule that describes set of strings associated to a token. Expressed as regular expression & describing how a particular token can be formed. The pattern matches each string in the set.

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Recognition of Tokens

Class Notes

Unit No: 2
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 8

Recognition of Tokens

→ Tokens obtained during lexical analysis are recognized by finite automata

→ FA is a simple machine that can be used to recognize patterns within input taken from alphabet. The primary task of FA is to accept or reject an input based on whether the defined pattern occurs within the input.

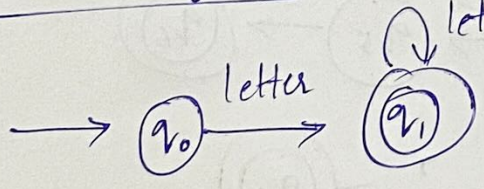
* Tokens are recognised with transition diagrams.

(1) Recognition of identifiers
(2) Recognition of delimiters
(3) Recognition of relational operator.
(4) Recognition of keywords such as if, else, for
(5) Recognition of numbers such as (int/float)

Identifiers!

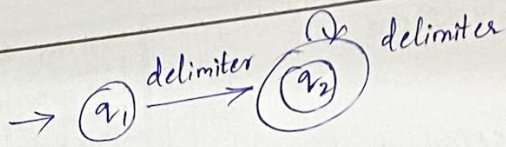
letter → a|b|c|---|z| A|B|---|Z
digit → 0|1|---|9

Transition Diagram



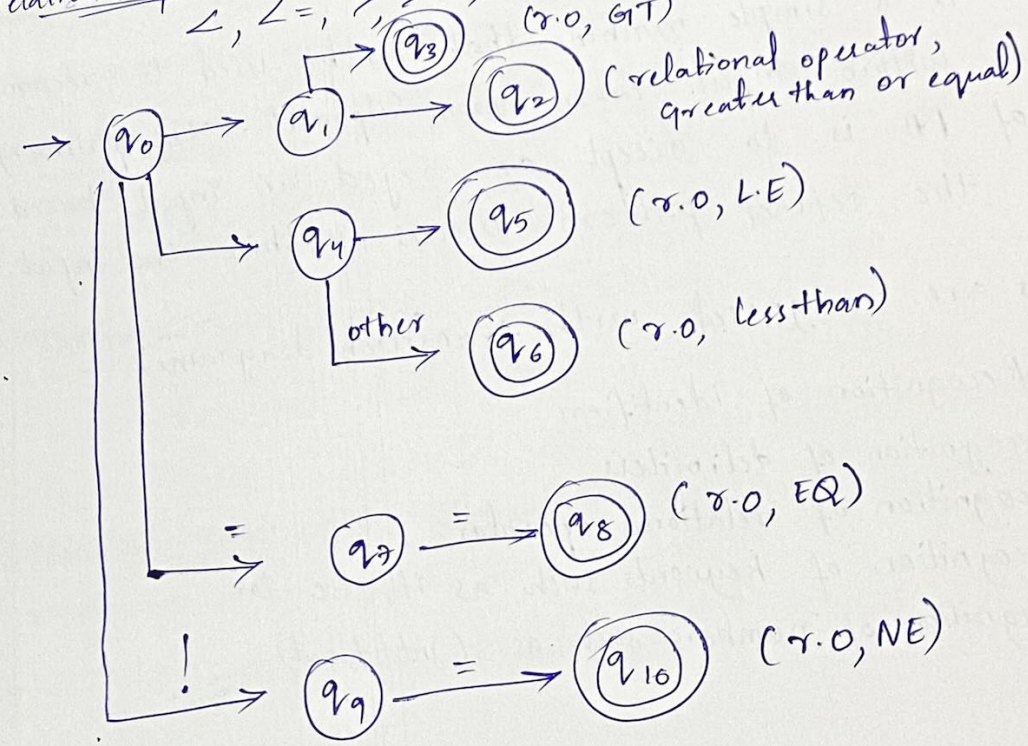
Delimiters! (Blankspace / Tab space / Newline character)

ws → delimiter (delimiter)*
↓
whitespace

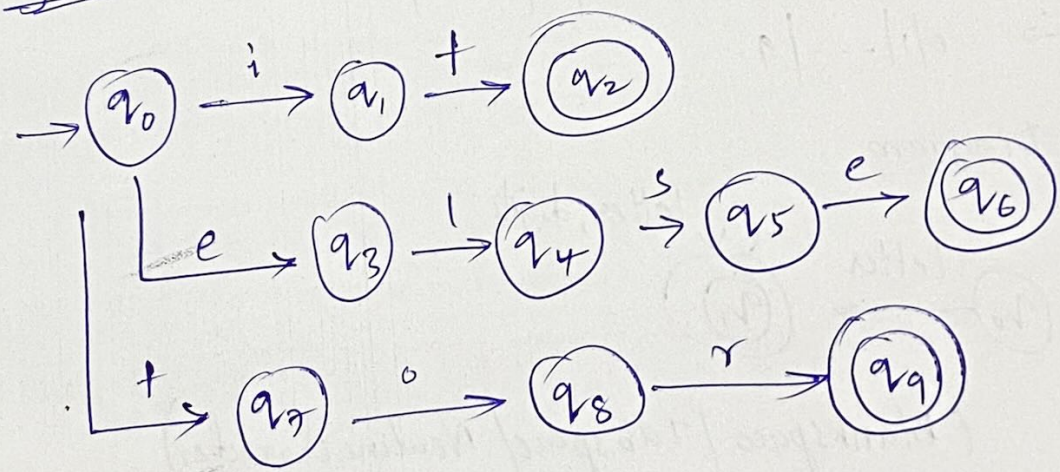


Relational operator:

<, <=, >, >=, ==, !=



Keywords:



Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Recognition of Tokens

Class Notes

Unit No: 2
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 9

Numbers

1 2 3

1 2 3 . 2 4

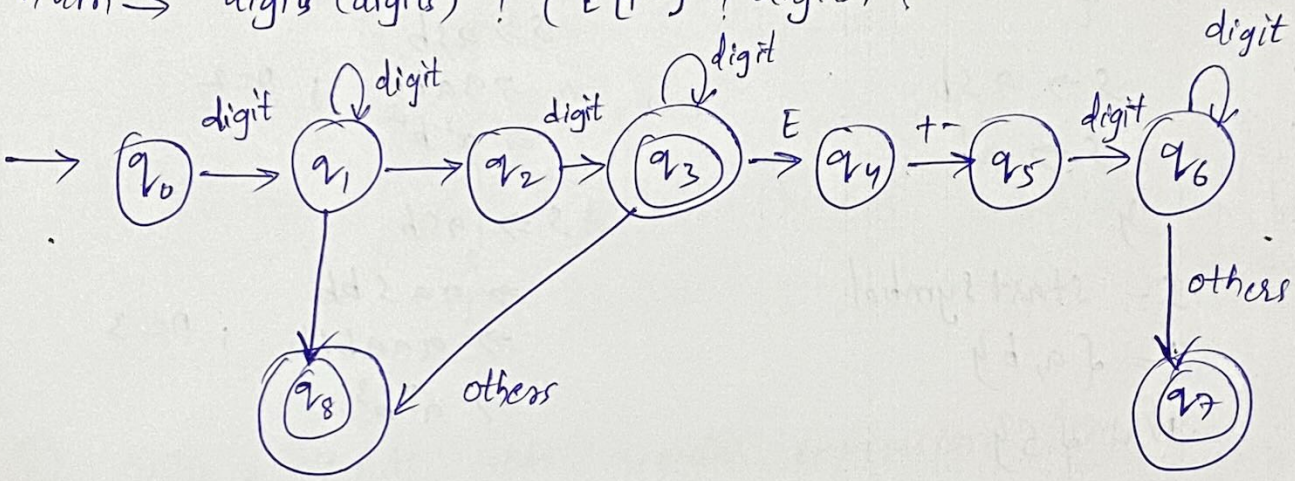
1 2 3 . 2 5 E 2 3

. 1 2 3 . 2 5 E - 2 3

digits → 0 | 1 | ... | 9

digits → digit (digits)*

num → digits (digits) ? (E [+-] ? digits) ?



Context free grammar

→ A grammar is said to be context free grammar 'G' = (N, T, P, S) where

N – Set of Non terminals
T – Set of terminals
P – Set of production rules
S – Start symbol

Example;

G: (N, T, P, S)

P: {
S → aSb
S → ab
}

S – start symbol
T = {a, b}
N = {S}

∴ L(G) = $a^n b^n$

Derivation:

$S \Rightarrow ab$; n = 1

$S \Rightarrow aSb$
$\Rightarrow aabb$; n = 2
$\Rightarrow a^2 b^2$

$S \Rightarrow aSb$
$\Rightarrow aaSbb$
$\Rightarrow aaabbb$; n = 3
$\Rightarrow a^3 b^3$

① Write a context free grammar to generate the language $L = \{ wcw^R / w \in \{a,b\}^* \}$ where c is a terminal symbol.

G = (N, T, P, S)
N = (S)
T = {a, b, c}
S – start symbol

P = {
S → aSa
S → bSb
S → c
}

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Context free Grammar.

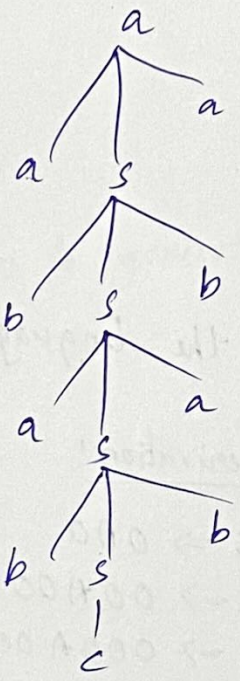
Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 10

Derivation!

$S \Rightarrow asa$
$\Rightarrow absba$
$\Rightarrow ababa$
$\Rightarrow ababsbaba$
$\Rightarrow ababcbaba$

$w = abab$
$w^R = baba$
$wcw^R = ababcbaba$
$w^R \rightarrow$ Reverse of w.



↓ Syntax / Parse tree.

(2) Write a CFG for regular expression $(0+1)^*01^*$

Derivation!

$S \Rightarrow AOB$
$\Rightarrow 0AOB$
$\Rightarrow 01AOB$
$\Rightarrow 010AOB$
$\Rightarrow 010\varepsilon OB$
$\Rightarrow 01001B$
$\Rightarrow 010011$

$G = \{N, T, P, S\}$
$V = \{S, A, B\}$
$T = \{0, 1\}$
S - start symbol

$P = \{ S \rightarrow AOB$
$A \rightarrow \varepsilon \mid 0A \mid 1A$
$B \rightarrow \varepsilon \mid 1B$
$\}$

(3) Write a CFG to generate strings of balanced paranthesis (or) well formed strings of paranthesis

P: {
S → ( )
S → (S)
S → SS
}

(or)

P: {
S → asb
S → ab
S → SS
}

$G_1 = \{N, T, P, S\}$

N = {S}

T = {(,)} or {a, b}

S = start symbol

(4) Write a CFG to generate the language $L = \{0^m 1^n 0^{m+n} / m, n \geq 1\}$

P: {
S → 0A0
A → 0A0 | 1B0
B → 1B0 | ε
}

Derivation:

S ⇒ 0A0
→ 00A00
→ 000A000
⇒ 0001B0000
→ 00111B000000
→ 00011ε00000.

$G_1 = \{N, T, P, S\}$

N = {S, A, B}

T = {0, 1}

S = start symbol.

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Context free grammar

Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 11

(5) Write a CFG to generate all strings {a, b} that are palindromes

$G = \{N, T, P, S\}$
$N = \{S\}$
$T = \{a, b\}$
S → start symbol.

P: {
$S \to aSa$
$S \to bSb$
$S \to a$
$S \to b$
$S \to \varepsilon$
}

(6) Write a CFG to generate the language $L = \{a^p b^q c^m / q = p+m\}$

$G = \{N, T, P, S\}$
$N = \{S, A, B\}$
$T = \{a, b\}$
S → start symbol

P: {
$S \to AB$
$A \to aAb / \varepsilon$
$B \to bBc / \varepsilon$
}

(7) Write a CFG to generate the language $L = \{a^m b^n / m \neq n\}$

$G = \{N, T, P, S\}$
$N = \{S, A, B\}$
$T = \{a, b\}$
S → start symbol

P: {
$S \to aSb / aA / bB$
$A \to aA / \varepsilon$
$B \to bB / \varepsilon$
}

(3) Write a CFG to generate strings of balanced parenthesis (or) well formed strings of parenthesis

(8) Write a CFG to generate a language $L(G) = \{a^n b^m \mid m, n \geq 1\}$

$G = \{N, T, P, S\}$
$N = \{S, A\}$
$T = \{a, b\}$
$S \rightarrow$ Start symbol

P: {
$S \rightarrow aS$
$S \rightarrow aA$
$A \rightarrow bA$
$A \rightarrow b$
}

(9) Construct a CFG to generate a language
$L(G) = \{a^m b^n c^m d^m \mid m, n \geq 1\}$

$G = \{N, T, P, S\}$
$N = \{S, A, B\}$
$T = \{a, b, c, d\}$

P: {
1. $S \rightarrow AB$
2. $A \rightarrow aAb \mid ab$
3. $B \rightarrow cBd \mid cd$
}

(10) Write a CFG to generate a language $L = \{a^n \mid n \geq 1\}$

$G = \{N, T, P, S\}$
$N = \{S\}$
$T = \{a\}$
$S \rightarrow$ Start symbol

P: {
$S \rightarrow aS$
$S \rightarrow a$
}

(11) Construct a CFG for the language
$L = \{a^n b^m c^m d^n \mid m, n \geq 1\}$

P: {
$S \rightarrow aAd$
$A \rightarrow aAd \mid ad$
$b \rightarrow bBC \mid bc \mid \varepsilon$
}

(or)

P: {
$S \rightarrow asd$
$S \rightarrow aAd$
$A \rightarrow bAc \mid bc$
}

$G = \{N, T, P, S\}$
$N = \{S, A\}$
$T = \{a, b, c, d\}$
S - Start Symbol

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Context free grammar

Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 12.

(12) Write a CFG for the language

$L = \{ a^{2n} / n \geq 1 \}$

P: {
$S \to aA$
$A \to aS$
$A \to a$
}

$G = \{N, T, P, S\}$
$N = \{S, A\}$
$T = \{a\}$
$S =$ Start Symbol.

(13) Write a CFG to generate the language

$L(G) = \{ (ab)^n / n \geq 1 \}$

P: {
$S \to aA$
$A \to bS$
$A \to a$
}

$G = \{N, T, P, S\}$
$N = \{S, A\}$
$T = \{a, b\}$
S - Start Symbol.

(14) Write a CFG to generate the language

$L = \{ (abc)^n / n \geq 1 \}$

P: {
$S \to aA$
$A \to bB$
$B \to cS$
$S \to c$
}

(or)

$S \to abA$
$A \to cS$
$A \to c$

$G = \{N, T, P, S\}$
$N = \{S, A, B\}$
$T = \{a, b, c\}$
S - Start Symbol.

thesis

(17). Write a context free grammar to generate the language.

$L = \{ a^n b^{2n} / n \geq 1 \}$

P: {
$S \rightarrow aSbb$
$S \rightarrow ab$
}

$G = \{ N, T, P, S \}$
$N = \{ S \}$
$T = \{ a, b \}$
S - start symbol

n+n/

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Derivation and Parsing

Class Notes

Unit No: 2
Lecture No:
Link to Session Planner (SP):
Book Reference:
Date Conducted:
Page No: 13

Derivation:

In the process of generating a language from the given production rules of a grammar, the non-terminals are represented by the corresponding strings of the right hand side of the production. But if there are more than 1 non-terminal which of the one's will be replaced must be determined. Depending on this selection, the derivation is divided into 2 types they are:

1. Left most derivation (LMD)
2. Right most derivation (RMD)

Left Most Derivation (LMD)

A derivation is called a left most derivation if we replace only the left most non-terminal by some production rule at each step of the generating process of the language from the grammar.

Right Most Derivation (RMD)

A derivation is called a Right most derivation if we replace only the right most non-terminal by some production rule at each step of the generating process of the language from the grammar.

Eg: Construct the string 0100110 from the following grammar by using leftmost & Right most derivation.

LMP

$S \Rightarrow 0S$
$\Rightarrow 01AA$
$\Rightarrow 010BA$
$\Rightarrow 0100BBA$
$\Rightarrow 01001BA$
$\Rightarrow 0100110A$
$\Rightarrow 0100110.$

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Parsetrees

Class Notes

Unit No:
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 14

Parse Tree

→ Parsing a string is finding a deviation for that string from a given grammar.

→ A parse tree is a tree representation of deriving a CFL (context free language). This type of trees are sometimes called as derivation trees.

→ A parse tree is an ordered tree in which the LHS of a production represents a parent node & RHS of a production represents a child node.

→ There are certain conditions for constructing a parse tree

i) Each vertex of a tree must have a label. The label is a NT or T or null.

ii) The root of the tree is the start symbol.

iii) The label of the internal vertices is a NT symbol.

iv) A vertex is called a leaf of the parse tree if its label is a T symbol.

→ find the parse tree for generating the string 11001010 for following grammar.

1. $S \rightarrow 1B \mid 0A$
2. $A \rightarrow 1 \mid 1S \mid 0AA$
3. $B \rightarrow 0 \mid 0S \mid 1BB$

Derivation

LMD:

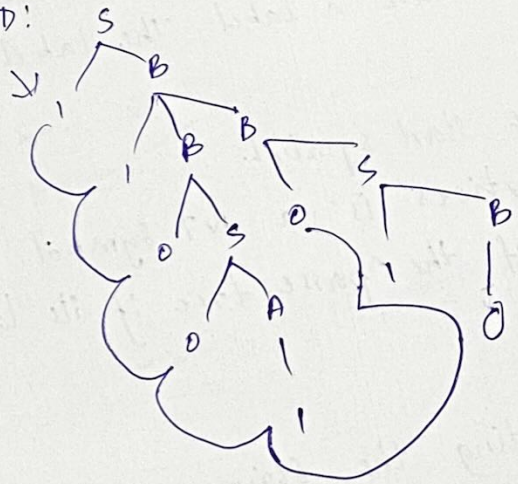
$S \Rightarrow 1B$
$\Rightarrow 11\underline{B}B$
$\Rightarrow 110\underline{S}B$
$\Rightarrow 1100\underline{A}B$
$\Rightarrow 11001B$
$\Rightarrow 1100105$
$\Rightarrow 11001018$
$\Rightarrow 11001010.$

RMD:

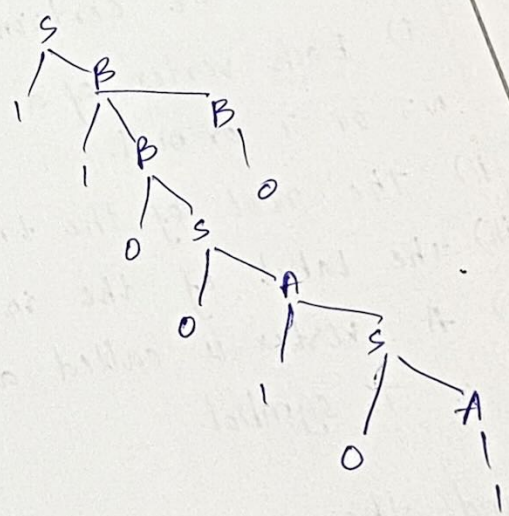
$S \Rightarrow 1B$
$\Rightarrow 11\underline{B}B$
$\Rightarrow 11B0$
$\Rightarrow 1105O$
$\Rightarrow 1100A0$
$\Rightarrow 110015O$
$\Rightarrow 11001A0$
$\Rightarrow 11001010.$

Parse tree

LMD:



RMD:



Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Ambiguity

Class Notes

Unit No:
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 15

Ambiguity in context free grammar.

→ A CFG 'G', such that same string has 2 parse trees is said to be ambiguous.

→ If there exist 2 or more LMD's or 2 or more RMD's to derive a string from a grammar is an ambiguous.

Eg: Consider CFG

$G = (N, T, P, S)$

$N = \{S\}$

$T = \{a, b, +, *\}$

S → Start symbol

P: {

1. $S \to S * S$
2. $S \to S + S$
3. $S \to a$
4. $S \to b$

}

Derivations: [a+b*a]

LMD

① $S \Rightarrow S * S$
$S \Rightarrow S + S * S$
$S \Rightarrow a + S * S$
$S \Rightarrow a + b * S$
$S \Rightarrow a + b * a$

② $S \Rightarrow S + S$
$S \Rightarrow S + S * S$
$S \Rightarrow a + S * S$
$S \Rightarrow a + b * S$
$S \Rightarrow a + b * a$

There are 2 LMD's

∴ Given grammar CFG G is ambiguous.

→ Prove that given grammar is ambiguous

$S \to as \mid AS \mid A$

$A \to As \mid a$

String: aaa

Derivation:

LMD

① $S \Rightarrow aS$
$\Rightarrow aaS$
$\Rightarrow aaA$
$\Rightarrow aaa$

② $S \Rightarrow A$
$\Rightarrow AS$
$\Rightarrow AAS$
$\Rightarrow aAS$
$\Rightarrow aaS$
$\Rightarrow aaA$
$\Rightarrow aaa$

There are 2 LMDs

∴ Given CFG is ambiguous.

→ Prove that the following grammar is ambiguous

$N = \{S\}$
$T = \{id, +, *\}$
$P: \{$
$S \rightarrow S+S \mid S*S \mid id$
$S = \{S\}$

string: [id+id * id]

Derivation:

LMD

① $S \Rightarrow S*S$
$\Rightarrow S+S*S$
$\Rightarrow id+S*S$
$\Rightarrow id+id*S$
$\Rightarrow id+id*id.$

$S \Rightarrow S+S$
$\Rightarrow S+S*S$
$\Rightarrow id+S*S$
$\Rightarrow id+id*S$
$\Rightarrow id+id*id.$

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Elimination of left recursion.

Class Notes

Unit No:
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 16

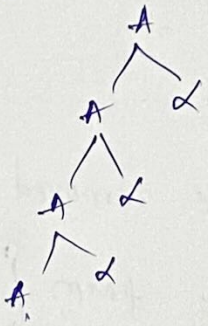
Left recursion & Elimination of left recursion

Left recursion.

→ A grammar is LR if it has a NT 'A' such that $A \xrightarrow{+} A$ for substring 'α'.

$A \to A\alpha$
$\Rightarrow A\alpha\alpha$
$\Rightarrow A\alpha\alpha\alpha$

⇒



→ A CFG is called left recursive if a NT 'A' as a left most symbol appears alternatively at the time of derivation, either immediately (called direct LR) or through some other NT definition (indirect LR).

Direct LR:

→ Let the grammar be $A \to A\alpha / \beta$ where α, β consists of NT's and are T's but β does not start with 'A'.

→ At the time of derivation if we start from 'A' and replaced by A by the production $A \to A\alpha$ then for all the times A will be the left most NT symbol.

Indirect LR:

→ Take a grammar in the form $A \to Ba \mid \lambda$, $B \to A\beta \mid \delta$, here β, λ, δ consists of NT's & are T's

→ If replace B by the production $B \to A\beta$ then A will appear again as left most NT, this is called the indirect LR.

Removal of LR in CFG:

→ Immediate LR can be removed by the following process.

→ for a grammar in the form $A \to A\alpha \mid \beta$

→ The equivalent grammar after removing LR becomes

$$A \to \beta A'$$
$$A' \to \alpha A' \mid \varepsilon$$

[∵ A' is new NT]

→ In general for a grammar in the form

$A \to A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$, where $\beta_1, \beta_2 \dots \beta_n$ does not start with 'A'.

→ the equivalent grammar after removing LR is

$$A \to \beta_1 A' \mid \beta_2 A' \mid \dots \beta_n A'$$
$$A' \to \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \varepsilon$$

Eg: Remove the LR from the following grammar

$E \to E+T \mid T$
$T \to T*F \mid F$
$F \to id \mid (E)$

Subject: ACD
Faculty: Bolleddu Devananda Rao
Topic: Elimination of leftfactoring

Class Notes

Unit No:
Lecture No:
Link to Session
Planner (SP):
Book Reference:
Date Conducted:
Page No: 17

Elimination of LR.

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow id \mid (E)$

Elimination of Left factoring

If RHS of more than one production starts with the same symbol, then such a grammar is called as Grammar with common prefixes.

· Eg: $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \alpha\beta_3$

→ This kind of grammar creates a problematic situation for top down parsers.

→ Top down parsers cannot decide which production must be chosen to parse the string in hand.

→ To remove this confusion, we use left factoring

Left Factoring :

→ It is a process by which the grammar with common prefixes is transformed to make it useful for top down parsers in left factoring

→ we make one production for each common prefixes.

→ The common prefix may be a terminal or a non-terminal or a combination of both.

→ Rest of the derivation is added by new productions.

→ The grammar obtained after the process of left factoring is called as left factored grammar.

$A \rightarrow \alpha\alpha_1 \mid \alpha\alpha_2 \mid \alpha\alpha_3$

Grammar with common prefixes.

left factoring →

$A \rightarrow \alpha A'$

$A' \rightarrow \alpha_1 \mid \alpha_2 \mid \alpha_3$

left factorial grammar.

Eg: Do left factoring in the following grammar.

$S \rightarrow \underbrace{i\,E\,t\,S}_{\alpha} \underbrace{\uparrow}_{\varepsilon(\beta_1)} \mid \underbrace{i\,E\,t\,S\,e\,S}_{\alpha} \underbrace{\uparrow}_{\beta_2} \mid a$

The left factored grammar is

$S \rightarrow i\,E\,t\,S\,S' \mid a$

$S' \rightarrow e\,S \mid \varepsilon$

$E \rightarrow b.$